

Setting up the Role-based Strategy plugin

By default, Jenkins has very basic security. To unlock professional-grade access control, you must install the *Role-based Authorisation Strategy* plugin.

- Navigate to *Manage Jenkins > Plugins*.
- Search for ‘*Role-based Authorization Strategy*’ and install it.
- Once installed, you must tell Jenkins to use this plugin. Go to *Manage Jenkins > Security*.
- Under the *Authorization* section, select the radio button for *Role-Based Strategy* and click *Save*.

Creating your ‘team’ users

Security is best demonstrated with different accounts. We will use Jenkins’ internal user database to create three distinct users for our simulation.

To create these, go to *Manage Jenkins > Users > Create User*:

- **admin:** This is your superuser who will have full control over the server.
- **dev-user:** Our target for the Least Privilege principle. This user should only see the ‘Alpha’ project.
- **ops-team:** A user representing the operations team who needs to see and manage all projects across the board.

 **Note:** At this stage, do not worry about permissions. By default, new users may see everything or nothing depending on your initial setup. We will fix this in the RBAC section.

Setting up the Role-based Strategy plugin

Jenkins does not come with advanced role management out of the box. To get started, you need to install the *Role-based Authorization Strategy* plugin.

- Go to *Manage Jenkins > Plugins* and install the plugin.
- Once installed, go to *Manage Jenkins > Security*.
- Under *Authorization*, switch to *Role-Based Strategy* and save.

Designing roles for developers and operations

Before assigning permissions to your users, you need to define the roles.

Think of a role as a ‘permission bucket’ that you will later drop users into.

For our scenario, we will design three distinct roles to cover our *admin*, *ops-team*, and *dev-user* accounts:

- **The Admin role:** This is the ‘full access’ role. It should have the *Administer* permission checked, giving Admin the ability to change any setting on the server.
- **The Ops role:** This is an operational role. It needs permission to manage jobs, view logs, and run builds, but it should not be able to change core system security or manage plugins.
- **The Developer role:** This is a restricted role. It is designed to give the *dev-user* access to only their specific project folder while keeping other projects invisible.

The two layers of RBAC

In this plugin, roles are split into two main tables:

Global roles: These define high-level access, such as whether a user can even log in to the Jenkins dashboard.

Item roles: These are project specific. They use Regular Expressions (Regex) to filter which jobs a user can see and interact with.

How to create global roles

Global roles are the foundation of your security setup. Without a global role, a user cannot even access the Jenkins dashboard, regardless of what other permissions they have.

- Navigate to *Manage Jenkins > Manage and Assign Roles > Manage Roles*.
- Under the *Global roles* section, we will create two roles:
 - a. **admin:** Type ‘admin’ in the *Role to add* box. Check the box for *Administer*. This gives full control over everything.
 - b. **read-only:** Type ‘read-only’ in the box. Check ONLY the *Overall/Read* permission.
- **The ‘read-only’ logic:** This role is essential for our *dev-user* and *ops-team*. It allows them to log in and see the general Jenkins layout without giving them permission to see or run any jobs yet.

The global role options are shown in Figure 1.



Figure 1: Global roles